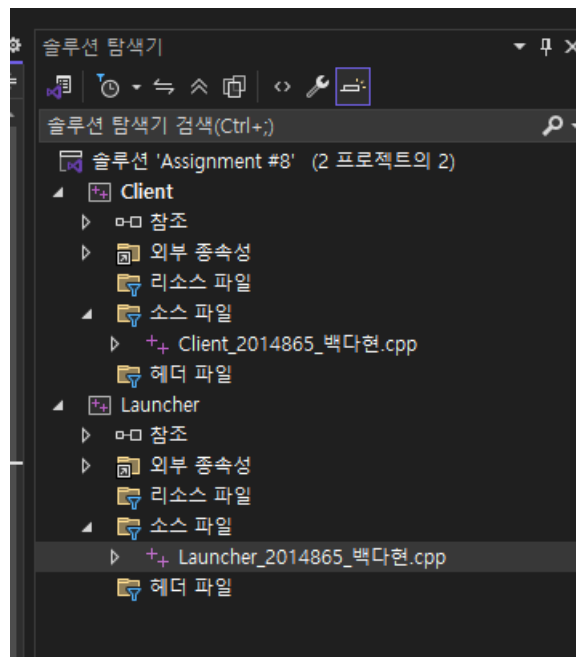


과제 #8

전체 큰 흐름

1. 새 솔루션 + Launcher/Client 콘솔 프로젝트 생성
2. Client.exe
 - a. 에코(echo) 기능만 먼저 구현
 - b. → “디버깅 중인지 체크” 추가
 - c. → “부모 프로세스가 Launcher.exe인지 체크” 추가
3. Launcher.exe
 - a. “자기 자신이 디버깅 당하는지” 탐지 코드(과제 7 재활용)
 - b. Client 프로세스를 실행하는 코드(CreateProcess)
 - c. Client를 “디버깅하는” 코드 (디버그 이벤트 루프)
4. 최종 연결/테스트
 - a. Client를 직접 실행하면 실패하는지
 - b. Launcher를 디버깅하면 실패하는지
 - c. Launcher가 Client를 잘 디버깅하는지

1단계: 솔루션 + 프로젝트 생성



솔루션 이름 : Assignment #8

각 프로젝트 이름 : Client , Launcher

2. Client.exe

2-A단계: Client Echo 기능 구현

```
// Client_2014865_백다현.cpp
#include <windows.h>
#include <stdio.h>
#include <string.h>

int main() {
    char buf[256]; // 사용자가 입력할 문자열 버퍼

    printf("=== Client Echo Program ===\n");
    printf("문자열을 입력하면 그대로 다시 출력합니다.\n");
    printf("프로그램을 종료하려면 \"exit\" 를 입력하세요.\n\n");

    while (1) {
        printf("> ");

        // 한 줄 입력 받기 (공백 포함)
        if (fgets(buf, sizeof(buf), stdin) == NULL) {
            // EOF나 입력 오류 시 루프 종료
            printf("\n[Client] 입력 오류 또는 EOF. 프로그램을 종료합니다.\n");
            break;
        }

        // 끝에 붙은 개행 문자(\n) 제거
        size_t len = strlen(buf);
        if (len > 0 && buf[len - 1] == '\n') {
            buf[len - 1] = '\0';
        }

        // 종료 명령 처리
        if (strcmp(buf, "exit") == 0) {
            printf("[Client] 종료 명령을 받았습니다. 프로그램을 종료합니다.\n");
            break;
        }

        // 에코 출력
        printf("echo: %s\n", buf);
    }
}
```

```
return 0;
}
```

```
Microsoft Visual Studio 디버그
=== Client Echo Program ===
문자열을 입력하면 그대로 다시 출력합니다.
프로그램을 종료하려면 "exit" 를 입력하세요.

> dd
echo: dd
> ff
echo: ff
> exit
[Client] 종료 명령을 받았습니다. 프로그램을 종료합니다.
```

2-B단계: 디버깅 여부 체크 함수 추가

```
#include <windows.h>
#include <stdio.h>
#include <string.h>

// 디버깅 여부 확인 함수
// - 현재 프로세스에 디버거가 붙어 있으면 true, 아니면 false
bool is_being_debugged() {
    BOOL bDebugged = FALSE;

    // 현재 프로세스를 대상으로 디버거 연결 여부 확인
    if (!CheckRemoteDebuggerPresent(GetCurrentProcess(), &bDebugged)) {
        // 실패한 경우, 일단 "디버깅 안 된다" 쪽으로 처리
        return false;
    }

    return (bDebugged == TRUE);
}

int main() {
    // 1) 프로그램 시작 시, 반드시 디버깅 중인지 먼저 확인
    if (!is_being_debugged()) {
        printf("[Client] Error: This program must be run under a debugger.\n");
        printf("[Client] 디버깅 상태가 아니므로 종료합니다.\n");
        Sleep(3000); // 메시지 보이도록 잠깐 대기
        return 1;
    }

    // 2) 여기부터는 에코 기능
    char buf[256]; // 사용자가 입력할 문자열 버퍼

    printf("=== Client Echo Program ===\n");
    printf("문자열을 입력하면 그대로 다시 출력합니다.\n");
```

```

printf("프로그램을 종료하려면 \"exit\" 를 입력하세요.\n\n");

while (1) {
    printf("> ");          // 프롬프트 표시

    // 한 줄 입력 받기 (공백 포함)
    if (fgets(buf, sizeof(buf), stdin) == NULL) {
        // EOF나 입력 오류 시 루프 종료
        printf("\n[Client] 입력 오류 또는 EOF. 프로그램을 종료합니다.\n");
        break;
    }

    // 끝에 붙은 개행 문자(\n) 제거
    size_t len = strlen(buf);
    if (len > 0 && buf[len - 1] == '\n') {
        buf[len - 1] = '\0';
    }

    // 종료 명령 처리
    if (strcmp(buf, "exit") == 0) {
        printf("[Client] 종료 명령을 받았습니다. 프로그램을 종료합니다.\n");
        break;
    }

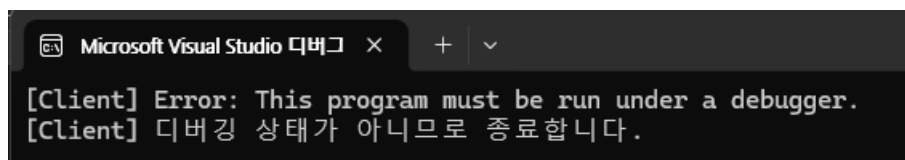
    // 에코 출력
    printf("echo: %s\n", buf);
}

return 0;
}

```

1) 디버거 없이 실행 테스트

- Client를 Startup Project로 설정
- Ctrl + F5 (Start Without Debugging)



→ 디버깅 상태가 아니므로 에코 루프까지는 가지 않고 바로 종료

2) 디버거로 실행해 보기

- Client를 Startup Project로 둔 상태에서
- F5 (Start Debugging)

```
Microsoft Visual Studio 디버그 x + v
=== Client Echo Program ===
문자열을 입력하면 그대로 다시 출력합니다.
프로그램을 종료하려면 "exit" 를 입력하세요.

> dd
echo: dd
> ff
echo: ff
> exit
[Client] 종료 명령을 받았습니다. 프로그램을 종료합니다.
```

→ 입력하면 `echo: ...` 정상 작동 → `exit` 입력 시 정상 종료

2-C단계. 부모 프로세스가 Launcher.exe인지 확인하는 함수 (`is_parent_launcher`) 추가

부모 프로세스 이름이 Launcher.exe 인지 확인하는 함수

```
// 2. 부모 프로세스 이름이 Launcher.exe 인지 확인하는 함수
bool is_parent_launcher() {
    DWORD myPid = GetCurrentProcessId();
    DWORD parentPid = 0;

    // (1) 전체 프로세스 스냅샷에서 "나 자신의 항목"을 찾아서 부모 PID 얻기
    HANDLE hSnap = CreateToolhelp32Snapshot(TH32CS_SNAPPROCESS, 0);
    if (hSnap == INVALID_HANDLE_VALUE) {
        return false;
    }

    PROCESSENTRY32 pe;
    pe.dwSize = sizeof(PROCESSENTRY32);

    if (Process32First(hSnap, &pe)) {
        do {
            if (pe.th32ProcessID == myPid) {
                parentPid = pe.th32ParentProcessID;
                break;
            }
        } while (Process32Next(hSnap, &pe));
    }

    CloseHandle(hSnap);

    if (parentPid == 0) {
        // 부모 PID를 못 찾았으면 실패로 처리
        return false;
    }
}
```

```

// (2) 다시 스냅샷을 떼서 parentPid에 해당하는 프로세스를 찾고, 이름 비교
hSnap = CreateToolhelp32Snapshot(TH32CS_SNAPPROCESS, 0);
if (hSnap == INVALID_HANDLE_VALUE) {
    return false;
}

pe.dwSize = sizeof(PROCESSENTRY32);
if (Process32First(hSnap, &pe)) {
    do {
        if (pe.th32ProcessID == parentPid) {
            bool ok = (_tcsicmp(pe.szExeFile, TEXT("Launcher.exe")) == 0);
            CloseHandle(hSnap);
            return ok;
        }
    } while (Process32Next(hSnap, &pe));
}

CloseHandle(hSnap);
return false;
}

```

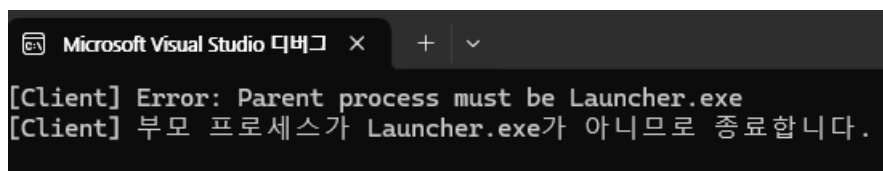
int main() 함수에 해당 함수 추가

```

int main() {
    // 1) 부모가 Launcher.exe 인지 먼저 검사
    if (!is_parent_launcher()) {
        printf("[Client] Error: Parent process must be Launcher.exe\n");
        printf("[Client] 부모 프로세스가 Launcher.exe가 아니므로 종료합니다.\n");
        Sleep(3000); // 메시지 보이도록 3초 기다리기
        return 1;
    }
}

```

1) 아직 Launcher를 안 만든 상태에서의 테스트



- F5로 실행 → 부모가 `devenv.exe` → `is_parent_launcher()` 에서 막힘 → 바로 에러 후 종료
- Ctrl+F5로 실행 → 부모가 `explorer.exe` → 역시 막힘 → 에러 후 종료

3. Launcher.exe

3-A 단계. 자기 자신이 디버깅 당하는지 탐지 코드 구현 (과제 7 로직 재활용)

```
#include <windows.h>
#include <intrin.h> // __readfsdword, __readgsqword
#include <stdio.h>
#include <debugapi.h> // DebugActiveProcess 등

// 전역 변수 (Watchdog 제어용)
volatile BOOL g_StopWatchdog = FALSE;
HANDLE g_hWatchdogThread = NULL;

// PEB 구조체: BeingDebugged 필드만 사용
typedef struct _PEB {
    BYTE Reserved1[2];
    BYTE BeingDebugged;
    BYTE Reserved2[1];
    PVOID Reserved3[2];
    PVOID Ldr;
    PVOID ProcessParameters;
} PEB, *PPEB;

// PEB를 통한 디버거 탐지
BOOL CheckDebuggerByPEB()
{
#ifdef _M_IX86
    PEB* peb = (PEB*)__readfsdword(0x30); // x86: TEB[0x30] → PEB
#elif _M_X64
    PEB* peb = (PEB*)__readgsqword(0x60); // x64: TEB[0x60] → PEB
#else
    return FALSE;
#endif
    return (peb->BeingDebugged != 0);
}

// 자기 자신 디버깅 여부 확인
BOOL IsDebuggingSelf()
{
    if (CheckDebuggerByPEB()) return TRUE;

    return FALSE;
}

// 주기적으로 디버깅 여부를 검사하는 Watchdog 스레드
DWORD WINAPI WatchdogThread(LPVOID lpParam)
{
    (void)lpParam;

    while (!g_StopWatchdog)
```

```

{
    if (IsDebuggingSelf())
    {
        printf("[WATCHDOG] Debugger detected! __fastfail()\n");
        __fastfail(1); // 즉시 프로세스 강제 종료
    }

    Sleep(300); // 0.3초마다 검사
}

printf("[WATCHDOG] 정상 종료\n");
return 0;
}

```

- 과제 7에서 사용했던 PEB 기반 디버깅 탐지 + 워치독 스레드 아이디어를 재활용

해당 코드 로직

1. PEB→BeingDebugged 값을 읽어서 디버거 여부 판단
2. 워치독 스레드가 0.3초마다 IsDebuggingSelf() 를 호출
3. 디버거 감지 시 곧바로 __fastfail(1) 호출 → 공격자가 디버깅을 지속할 수 없음

1) F5로 실행 (VS 디버깅 실행)

```

=== Launcher start ===
[Launcher] Error: Launcher must not be debugged.
[WATCHDOG] 정상 종료

```

→ 바로 Watchdog이 디버깅을 감지 → → 프로그램 즉시 종료 (VS 예외 팝업)

2) Ctrl+F5으로 실행

```

=== Launcher start ===

C:\Users\백다현\source\repos\Client\x64\Debug\
이 창을 닫으려면 아무 키나 누르세요...|

```

→ Watchdog 정상 실행 → [WATCHDOG] 정상 종료 → 출력 없음 (계속 running 상태) → 아직 main에서 아무 작업 없이 종료됨

3-B 단계. Client 프로세스를 실행하는 코드 (CreateProcessA) 테스트

```

// Client 실행 함수
// - 성공 시 Client의 PID를 out 매개변수로 반환
BOOL StartClientProcess(DWORD* pClientPid)
{

```



```

STARTUPINFOA si = { 0 };
PROCESS_INFORMATION pi = { 0 };
si.cb = sizeof(si);

if (!CreateProcessA(
    "Client.exe", // lpApplicationName
    NULL,        // lpCommandLine
    NULL, NULL, FALSE,
    0,          // 디버그 플래그 없이 일반 실행
    NULL, NULL,
    &si, &pi
)) {
    printf("[Launcher] Failed to start Client.exe. error=%lu\n", GetLastError());
    return FALSE;
}

*pClientPid = pi.dwProcessId;

// 핸들은 더 이상 사용하지 않으므로 닫아준다.
CloseHandle(pi.hThread);
CloseHandle(pi.hProcess);

printf("[Launcher] Client started. pid=%lu\n", *pClientPid);
return TRUE;
}

```

- `CreateProcessA` 를 사용하여 Client.exe를 자식 프로세스로 실행하였고, 이때 생성된 PID를 이후 디버깅용으로 사용하였다

1) Ctrl+F5으로 실행

```

=== Launcher start ===
[Launcher] Client started. pid=10908
[Client] Error: Parent process must be Launcher.exe
[Client] 부모 프로세스가 Launcher.exe가 아니므로 종료합니다.

```

3-C 단계. Client를 “디버깅하는” 코드 (디버그 이벤트 루프)

- `DebugActiveProcess(clientPid)` 로 Client에 디버거로 붙는다.
- `WaitForDebugEvent()` 로 디버그 이벤트를 기다린다.
- 특별한 처리 없이 항상 `DBG_CONTINUE` 로 `ContinueDebugEvent()` 를 호출한다.
- `EXIT_PROCESS_DEBUG_EVENT` 가 발생하면 루프를 종료한다.

```

// Client 프로세스를 디버깅하는 함수
BOOL DebugClient(DWORD clientPid)
{
    if (!DebugActiveProcess(clientPid)) {
        printf("[Launcher] DebugActiveProcess failed. error=%lu\n", GetLastError());
        return FALSE;
    }

    printf("[Launcher] Debugging client pid=%lu\n", clientPid);

    DEBUG_EVENT dbgEvent;
    BOOL running = TRUE;

    while (running)
    {
        // 디버그 이벤트 대기
        if (!WaitForDebugEvent(&dbgEvent, INFINITE)) {
            printf("[Launcher] WaitForDebugEvent failed. error=%lu\n", GetLastError());
            break;
        }

        DWORD continueStatus = DBG_CONTINUE;

        switch (dbgEvent.dwDebugEventCode)
        {
            case EXIT_PROCESS_DEBUG_EVENT:
                printf("[Launcher] Client process exited. code=%lu\n",
                    dbgEvent.u.ExitProcess.dwExitCode);
                running = FALSE;
                break;

            default:
                break;
        }

        if (!ContinueDebugEvent(dbgEvent.dwProcessId,
                                dbgEvent.dwThreadId,
                                continueStatus))
        {
            printf("[Launcher] ContinueDebugEvent failed. error=%lu\n", GetLastError());
            break;
        }
    }

    // Client는 이미 종료된 상태. Detach는 선택적
    DebugActiveProcessStop(clientPid);
}

```

```
    return TRUE;
}
```

최종 int main()

```
int main()
{
    printf("=== Launcher start ===\n");

    // (1) Watchdog 스레드 시작 (자기 자신 디버깅 탐지)
    g_hWatchdogThread = CreateThread(
        NULL, 0,
        WatchdogThread,
        NULL, 0, NULL
    );

    if (g_hWatchdogThread == NULL) {
        printf("[Launcher] Failed to create watchdog thread. error=%lu\n", GetLastError());
        return 1;
    }

    // 시작 직후 한 번 더 디버깅 여부 확인
    if (IsDebuggingSelf()) {
        printf("[Launcher] Error: Launcher must not be debugged.\n");
        g_StopWatchdog = TRUE;
        WaitForSingleObject(g_hWatchdogThread, INFINITE);
        CloseHandle(g_hWatchdogThread);
        return 1;
    }

    // (2) Client 프로세스 실행
    DWORD clientPid = 0;
    if (!StartClientProcess(&clientPid)) {
        g_StopWatchdog = TRUE;
        WaitForSingleObject(g_hWatchdogThread, INFINITE);
        CloseHandle(g_hWatchdogThread);
        return 1;
    }

    // (3) Client 프로세스를 디버깅 (선택 디버거 역할)
    DebugClient(clientPid);

    // (4) 종료 시 워치독 스레드 정리
    g_StopWatchdog = TRUE;
    WaitForSingleObject(g_hWatchdogThread, INFINITE);
    CloseHandle(g_hWatchdogThread);
}
```

```
printf("[Launcher] graceful shutdown.\n");
return 0;
}
```

결과 테스트

```
=== Launcher start ===
[Launcher] Client started. pid=9940
[Launcher] Debugging client pid=9940
[DEBUG] Parent Name = Launcher.exe
=== Client Echo Program ===
문자열을 입력하면 그대로 다시 출력합니다.
프로그램을 종료하려면 "exit" 를 입력하세요.

> dd
echo: dd
> ff
echo: ff
> |
```

4단계. 최종 연결/테스트

4-a단계. Client를 직접 실행하면 실패하는지

```
[DEBUG] Parent Name = explorer.exe
[Client] Error: Parent process must be Launcher.exe
[Client] 부모 프로세스가 Launcher.exe가 아니므로 종료합니다.
```

4-b단계. Launcher를 디버깅하면 실패하는지

```
=== Launcher start ===
[Launcher] Error: Launcher must not be debugged.
[WATCHDOG] 정상 종료
```

4-c단계. Launcher가 Client를 잘 디버깅하는지

```
=== Launcher start ===  
[Launcher] Client started. pid=9940  
[Launcher] Debugging client pid=9940  
[DEBUG] Parent Name = Launcher.exe  
=== Client Echo Program ===  
문자열을 입력하면 그대로 다시 출력합니다.  
프로그램을 종료하려면 "exit" 를 입력하세요.  
  
> dd  
echo: dd  
> ff  
echo: ff  
> |
```